



Teknoloji Fakültesi

BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

Yapay Zeka Destekli Agent ile 3B Modelleme

BİTİRME PROJESİ

Bilgisayar Mühendisliği Bölümü

DANIŞMAN

Prof. Dr. ŞAHİN UYAYER

İSTANBUL, 2026

MARMARA ÜNİVERSİTESİ
TEKNOLOJİ FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

Marmara Üniversitesi Teknoloji Fakültesi Bilgisayar Mühendisliği Öğrencileri Burak Yüceler (170422852), Abdurrahman Hamza Gündoğdu (170422012) ve Ali Batuhan Dağcı (170421962) tarafından **"Yapay Zeka Destekli Agent ile 3B Modelleme"** başlıklı proje çalışması, 11.06.2026 tarihinde savunulmuş ve jüri üyeleri tarafından başarılı bulunmuştur.

Jüri Üyeleri

	Görev	İmza
Prof. Dr. Şahin Uyaver — Marmara Üniversitesi	(Danışman)	
Doç. Dr. Ayşe Berna Altınel Girgin — Marmara Üniversitesi	(Üye)	
Dr. Öğr. Üyesi Neşe Özdemir — Marmara Üniversitesi	(Üye)	
Arş. Gör. Dr. Merve Pınar — Marmara Üniversitesi	(Üye)	

ÖNSÖZ

Proje çalışmamız süresince karşılaştığımız bütün problemlerde bilgi ve deneyimini bizimle paylaşan, yönlendirmeleriyle çalışmaya yön veren değerli danışmanımız sayın Prof. Dr. Şahin Uyaver'a en içten teşekkürlerimizi sunarız.

Bu proje çalışması; doğal dil ile üç boyutlu sahne kurgusunu birleştiren, büyük dil modelleri (LLM), çok-ajanlı orkestrasyon ve Blender otomasyonunu bir araya getiren bütünleşik bir sistemin tasarlanması ve geliştirilmesi amacıyla yürütülmüştür.

Proje çalışmamız sırasında maddi ve manevi desteklerini esirgemeyen ailelerimize ve değerli çalışma arkadaşlarımıza sonsuz teşekkürlerimizi sunarız.

İstanbul, 2026

Burak Yüceler

Abdurrahman Hamza Gündoğdu

Ali Batuhan Dağcı

İÇİNDEKİLER

Bölüm	Sayfa
1. GİRİŞ	12
1.1. Proje Çalışmasının Amacı ve Önemi	13
1.2. Problemin Tanımı	13
1.3. Kapsam ve Sınırlar	14
2. LİTERATÜR TARAMASI VE İLGİLİ ÇALIŞMALAR	15
2.1. Doğal Dilden 3B Üretimine Genel Bakış	15
2.2. Üretken Yöntemler: GAN ve CLIP Tabanlı Yaklaşımlar	15
2.3. Metinden 3B Sahne Üretimi ve Nesne–Mekân İlişkisi	15
2.4. Bu Çalışmanın Konumu ve Farkı	16
3. GEREKSİNİM ANALİZİ	17
3.1. Aktörler ve Kullanım Senaryoları	17
3.2. Fonksiyonel Gereksinimler	18
3.3. Fonksiyonel Olmayan Gereksinimler	18
4. SİSTEM MİMARİSİ VE TASARIM	19
4.1. Genel Mimari ve Katmanlı İşlem Hattı	19
4.2. v2 Katalog Şeması ve Semantik Katman	19
4.3. Çok-Ajanlı Tasarım Takımı	21
4.4. Yerleştirme ve Geometri Mantığı	22
4.5. Blender Eklentisi ve HTTP Köprüsü	24
4.6. Otomatik Kategorize İşlem Hattı	25
4.7. Web Tabanlı Kullanıcı Arayüzü	26
4.8. Teknoloji Yığını	28
5. GERÇEKLEŞTİRİM (UYGULAMA) DETAYLARI	29
5.1. Proje Dizin Yapısı	29
5.2. Kurulum ve Çalıştırma	30
5.3. Uçtan Uca Bir İsteğin Akışı (Sequence)	31
5.4. Kritik Kod Bileşenleri	31
6. BULGULAR VE TARTIŞMA	32
6.1. Test Yöntemi	32
6.2. Workflow A — Sıfırdan Oda Kurulumu	32
6.3. Workflow B — Hazır Ev Modeli İçinde Yerleştirme	33

Bölüm	Sayfa
6.4. Asset Ekleme ve Otomatik Kategorize Sonuçları	34
6.5. Niceliksel Değerlendirme	35
6.6. Bilinen Sınırlar	37
7. SONUÇLAR	38
7.1. Gelecek Çalışmalar	38
KAYNAKLAR	39

ÖZET

Yapay Zeka Destekli Agent ile 3B Modelleme

Bu proje çalışmasında, kullanıcının serbest biçimli doğal dil cümleleriyle ifade ettiği iç mekân tasarım isteklerini (örneğin "5x4 metrelik bir ofise endüstriyel masa ve sandalye koy") otomatik olarak üç boyutlu (3B) bir Blender sahnesine dönüştüren, çok-ajanlı yapay zekâ tabanlı bir sistem tasarlanmış, geliştirilmiş ve uygulanmıştır.

Sistem dört katmanlı bir işlem hattı (pipeline) üzerine kuruludur: (1) **Mekân Analisti (Space Analyst)** kullanıcı cümlesini çözümleyerek oda tipini, boyutlarını ve gerekli mobilya "slot"larını belirler; (2) **Mobilya Seçici (Furniture Selector)** anlamsal (semantik) etiketlerle zenginleştirilmiş bir 3B model kataloğundan her slot için en uygun varlığı seçer; (3) **Yerleşim Tasarımcısı (Layout Designer)** her mobilyanın konum ve dönüş açısını, oda sınırları ve nesneler arası ilişkileri (örneğin "masanın önündeki sandalye") gözeterek hesaplar; (4) **Blender Yürütücü (Blender Executor)**, hesaplanan sahneyi HTTP üzerinden Blender içine kurulan bir eklentiye göndererek oda ve mobilyaları içe aktarır.

Çalışmanın özgün katkıları arasında; varlık metadatasını anlamsal etiketler, oda uyumları, taban ayak izi (footprint) ve yönelim (forward axis) bilgileriyle genişleten **v2 katalog şeması**; mobilyaların oda ya da bina iç hacminin dışına taşmasını engelleyen **iç hacim sınırlayıcı kutu (interior bbox) kırpma** algoritması; ve yeni 3B modellerin kataloğa tutarlı biçimde eklenmesini sağlayan **hibrit otomatik kategorize** (dosya adı/malzeme örüntüsü + Blender ile başsız ölçüm + temel bileşen analizi + dil modeli ile zenginleştirme) yöntemi yer almaktadır.

Sistem, sıfırdan oda kurma (Workflow A) ve hazır bir bina/ev modeli içine yerleştirme (Workflow B) olmak üzere iki ana akışı desteklemektedir. Geliştirilen çözüm, doğal dil ile 3B sahne kurgusu arasındaki açığı kapatarak; iç mimari konseptlerinin hızlı prototiplenmesine olanak tanıyan, genişletilebilir ve büyük dil modeli sağlayıcısından bağımsız bir altyapı ortaya koymuştur.

Anahtar Kelimeler: Çok-ajanlı sistemler, büyük dil modelleri, doğal dil işleme, 3B sahne sentezi, Blender otomasyonu, iç mimari tasarım, model bağlam protokolü (MCP).

Haziran, 2026

Burak Yüceler

Abdurrahman Hamza Gündoğdu

Ali Batuhan Dağcı

ABSTRACT

3D Modeling with an AI-Powered Agent

In this project, a multi-agent, artificial-intelligence-based system was designed, developed and implemented that automatically converts free-form natural-language interior-design requests (e.g., "place an industrial desk and a chair in a 5x4 office") into a three-dimensional (3D) Blender scene.

The system is built on a four-layer pipeline: (1) the **Space Analyst** parses the user's sentence to determine the room type, dimensions and required furniture "slots"; (2) the **Furniture Selector** chooses, for each slot, the most suitable asset from a 3D-model catalog enriched with semantic tags; (3) the **Layout Designer** computes each item's position and rotation while respecting room boundaries and inter-object relationships (e.g., "the chair in front of the desk"); and (4) the **Blender Executor** sends the computed scene over HTTP to an add-on installed inside Blender to import the room and furniture.

The original contributions of the study include a **v2 catalog schema** that extends asset metadata with semantic tags, room compatibilities, footprints and forward-axis information; an **interior bounding-box clamping** algorithm that prevents furniture from extending beyond the room or building interior; and a **hybrid auto-categorization** method (filename/material pattern + headless Blender measurement + principal component analysis + LLM enrichment) for adding new 3D models to the catalog consistently.

The system supports two main workflows: building a room from scratch (Workflow A) and placing furniture inside a prebuilt building/house model (Workflow B). The resulting solution bridges the gap between natural language and 3D scene composition, providing an extensible, LLM-provider-independent infrastructure that enables rapid prototyping of interior-design concepts.

Keywords: Multi-agent systems, large language models, natural language processing, 3D scene synthesis, Blender automation, interior design, model context protocol (MCP).

June, 2026

Burak Yüceler

Abdurrahman Hamza Gündoğdu

Ali Batuhan Dağcı

SEMBOLLER VE NOTASYON

Sembol	Açıklama
(x, y, z)	Dünya koordinat sistemindeki konum bileşenleri (metre)
$W \times D \times H$	Genişlik $\times$ Derinlik $\times$ Yükseklik (width $\times$ depth $\times$ height)
rot_z	Z eksenini etrafındaki dönüş açısı (derece)
+Y / -Y / +X / -X	Modelin ön yön (forward axis) eksenini
bbox	Sınırlayıcı kutu (bounding box); min/max köşe çiftiyle tanımlanır
footprint	Mobilyanın zemindeki taban ayak izi (genişlik, derinlik + açıklık payları)
clearance_front / clearance_sides	Mobilyanın ön / yan tarafında bırakılması gereken boş alan (metre)
origin_offset	Oda yerel orijininin bina/dünya uzayına ötelenmesi [x, y, z]
interior_bbox_local	Bina/oda iç hacminin yerel koordinatlardaki sınırlayıcı kutusu

Koordinat eksenini anlaşması: X eksenini batı(-)/doğu(+), Y eksenini güney(-)/kuzey(+), Z eksenini yukarı. Dönüş açısı için $rot_z = 0$ modelin +Y yönüne, $rot_z = 180$ modelin -Y yönüne baktığı durumu ifade eder.

KISALTMALAR

Kısaltma	Açılım
YZ	Yapay Zekâ
LLM	Large Language Model (Büyük Dil Modeli)
NLP	Natural Language Processing (Doğal Dil İşleme)
MCP	Model Context Protocol (Model Bağlam Protokolü)
API	Application Programming Interface
HTTP	HyperText Transfer Protocol
JSON	JavaScript Object Notation
CLI	Command Line Interface (Komut Satırı Arayüzü)
UI	User Interface (Kullanıcı Arayüzü)
PCA	Principal Component Analysis (Temel Bileşen Analizi)
BBox	Bounding Box (Sınırlayıcı Kutu)
2B / 3B	İki Boyutlu / Üç Boyutlu
ASGI	Asynchronous Server Gateway Interface
HDRI	High Dynamic Range Image
SVG	Scalable Vector Graphics
FR / NFR	Functional Requirement / Non-Functional Requirement

ŞEKİL LİSTESİ

No	Açıklama	Sayfa
Şekil 1.1	Doğal dilden 3B sahneye dönüşümün kavramsal gösterimi	12
Şekil 3.1	Kullanım senaryosu (use-case) diyagramı	17
Şekil 4.1	Sistemin yüksek seviyeli işlem hattı (pipeline) akışı	19
Şekil 4.2	v2 katalog şemasının katmanları ve örnek varlık metadatası	21
Şekil 4.3	Çok-ajanlı takım etkileşim/akış diyagramı	22
Şekil 4.4	İç hacim sınırlayıcı kutu (interior bbox) kırpmaya görselleştirmesi	24
Şekil 4.5	Blender eklentisi N-paneli ve "Start Server" düğmesi	25
Şekil 4.6	auto_categorize hibrit kategorize işlem hattı	26
Şekil 4.7	Web arayüzü — "Tasarım" sekmesi	27
Şekil 4.8	Web arayüzü — "Yapboz" 2B yerleşim düzenleyici	27
Şekil 5.1	Proje dizin yapısı (ağaç görünümü)	29
Şekil 5.2	Kurulum: Blender eklentisinin yüklenmesi	30
Şekil 5.3	Uçtan uca istek için sıra (sequence) diyagramı	31
Şekil 6.1	Workflow A çıktısı — sıfırdan oluşturulan oda ve mobilyalar	33
Şekil 6.2	Workflow B çıktısı — bina modeli içinde yerleştirilmiş mobilyalar	34
Şekil 6.3	Ajan adımlarına ait çalışma günlüğü (log) ve JSON çıktıları	35
Şekil 6.4	add_asset.py otomatik kategorize terminal çıktısı	35

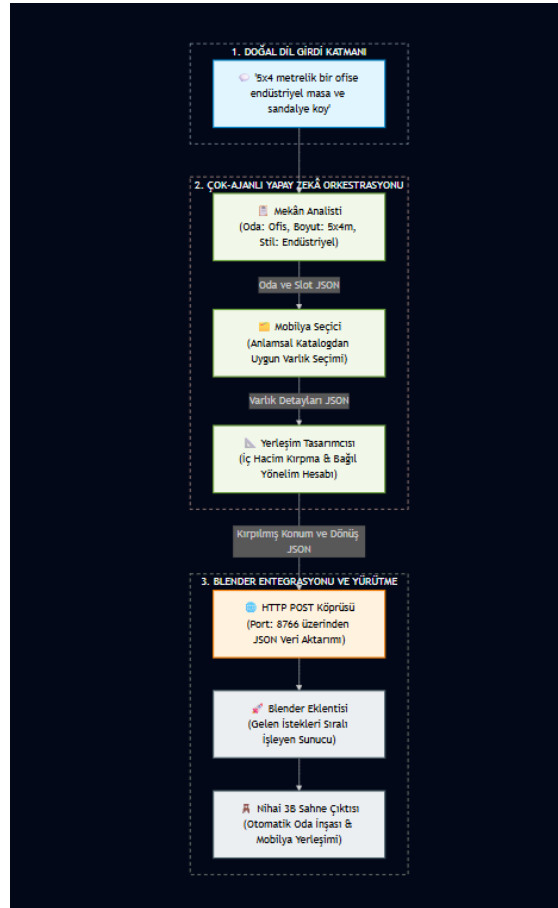
TABLO LİSTESİ

No	Açıklama	Sayfa
Tablo 2.1	İlgili çalışmalarla karşılaştırma	16
Tablo 3.1	Fonksiyonel gereksinimler	18
Tablo 3.2	Fonksiyonel olmayan gereksinimler	18
Tablo 4.1	v2 katalog şeması alanları	20
Tablo 4.2	Ajanlar: roller, modeller ve kullandıkları araçlar	21
Tablo 4.3	Desteklenen yerleştirme (placement) tipleri	23
Tablo 4.4	Blender eklentisi HTTP eylem (handler) listesi	24
Tablo 4.5	Otomatik kategorize alt-kategori eşleştirmeleri (örnek)	26
Tablo 4.6	Teknoloji yığını ve bağımlılıklar	28
Tablo 6.1	Mevcut katalog varlıkları	34
Tablo 6.2	Algoritma katmanı niceliksel ölçüm sonuçları	36
Tablo 6.3	Bilinen sınırlar ve etkileri	37

1. GİRİŞ

Üç boyutlu iç mimari tasarım süreçleri geleneksel olarak uzmanlık gerektiren, zaman alıcı ve genellikle özel yazılımlara (CAD/3B modelleme araçları) hâkimiyet isteyen süreçlerdir. Bir mekânın kurgulanması; oda geometrisinin oluşturulması, uygun mobilyaların seçilmesi, bu mobilyaların ölçek ve yönelimlerinin doğru ayarlanması ve birbirleriyle olan konumsal ilişkilerinin (örneğin bir sandalyenin masaya dönük olması) elle düzenlenmesi gibi pek çok adımı içerir. Bu adımların her biri, kullanıcının yazılımın iç mantığına ve 3B uzaydaki dönüşüm kavramlarına aşina olmasını gerektirir.

Son yıllarda büyük dil modellerinin (LLM) doğal dil anlama ve yapılandırılmış çıktı üretme yeteneklerindeki gelişme, kullanıcı niyetinin serbest metinden çıkarılıp makine tarafından işlenebilir komutlara çevrilmesini mümkün kılmıştır [1]. Bu çalışma, söz konusu yeteneği üç boyutlu sahne sentezi alanına taşıyarak [2-6]; kullanıcının yalnızca bir cümle yazarak iç mekân tasarımı yapabildiği bütünsel bir sistem önermektedir.



Şekil 1.1 Doğal dilden 3B sahneye dönüşümün kavramsal gösterimi.

1.1. Proje Çalışmasının Amacı ve Önemi

Bu proje çalışmasının amacı; doğal dil ile ifade edilen iç mekân tasarım isteklerini, insan müdahalesi olmadan otomatik olarak üç boyutlu bir Blender sahnesine dönüştüren, modüler ve genişletilebilir bir yapay zekâ sistemi geliştirmektir. Sistem, kullanıcı ile 3B tasarım yazılımı arasındaki teknik bilgi engelini ortadan kaldırarak iç mimari konseptlerinin hızlı prototiplenmesine olanak tanır.

Projenin önemi şu noktalarda toplanmaktadır:

- **Erişilebilirlik:** Kullanıcının 3B modelleme yazılımı bilgisine ihtiyaç duymadan, yalnızca metinle tasarım yapabilmesini sağlar.
- **Tutarlılık:** Mobilyaların ölçeği, yönelimi ve oda içindeki konumsal ilişkileri, elle ayar yerine kurallı ve anlamsal metadataya dayalı bir algoritma ile belirlenir.
- **Genişletilebilirlik:** Yeni 3B modeller, hibrit bir otomatik kategorize akışıyla kataloğa tutarlı biçimde eklenebilir.
- **Sağlayıcı bağımsızlığı:** Sistem, varsayılan olarak Google Gemini, alternatif olarak Anthropic Claude modelleriyle çalışabilecek biçimde tasarlanmıştır; tek bir sağlayıcıya bağımlı değildir.

Bu çalışma aynı zamanda daha geniş bir hedefin ara basamağı niteliğindedir: nihai amaç, yapay zekâ ajanlarının Blender ile **sıfırdan modelleme** yapabilmesidir. Mevcut proje, bu hedefe ulaşmadan önce **hazır modellerle iç dekorasyon** problemine odaklanarak hem altyapıyı hem de ajan orkestrasyon yöntemlerini olgunlaştırmaktadır.

1.2. Problemin Tanımı

Doğal dilden 3B sahne üretimi problemi, birbirini izleyen birkaç alt-problemin çözülmesini gerektirir:

1. **Niyet çıkarımı:** Serbest metinden oda tipi, boyutlar, stil ve istenen mobilyaların belirlenmesi.
2. **Varlık eşleştirme:** Belirlenen ihtiyaçların, mevcut bir 3B model kataloğundaki uygun varlıklarla eşleştirilmesi.
3. **Uzamsal yerleştirme:** Seçilen mobilyaların oda içinde çakışmadan, doğru ölçek ve yönelimle, anlamlı konumsal ilişkiler (önünde/yanında/karşısında) korunarak konumlandırılması.
4. **Sahne yürütme:** Hesaplanan yerleşimin gerçek bir 3B yazılımı (Blender) içinde inşa edilmesi.

Bu alt-problemlerin tek bir dev dil modeli isteminde (prompt) çözülmeye çalışılması; hata ayıklamayı zorlaştıran, tutarsız ve denetlenemez sonuçlar üretir. Bu nedenle çalışmada, her alt-problemi ayrı bir uzman ajanın üstlendiği **çok-ajanlı** bir yaklaşım benimsenmiştir [7].

1.3. Kapsam ve Sınırlar

Bu rapor, sistemin mevcut sürümüne — yani hazır 3B modellerle otomatik iç dekorasyona — odaklanmaktadır. Sistemin uzun vadeli "doğal dilden sıfırdan 3B modelleme" vizyonu bu sürümün kapsamı dışında olup yalnızca Bölüm 7.1 altında gelecek çalışma olarak ele alınmıştır.

Kapsam içi. Mevcut sürüm aşağıdaki yetenekleri kapsar:

- Serbest doğal dil isteğinden oda tipi, boyut, stil ve mobilya ihtiyaçlarının çıkarılması;
- Bu ihtiyaçların anlamsal katalogdan uygun varlıklarla (oda tipi/stil/uyumluluk filtreleriyle) eşleştirilmesi;
- Mobilyaların, oda sınırları ve bağlı konum ilişkileri (önünde/yanında/karşısında) gözetilerek otomatik yerleştirilmesi;
- Hesaplanan sahnenin Blender'da otomatik inşası (oda oluşturma + mobilya içe aktarma);
- Hem sıfırdan oda kurulumu (Workflow A) hem de hazır bir bina modelinin belirtilen odasına yerleştirme (Workflow B);
- Kullanıcının 2B kuşbakışı plan üzerinde mobilyaları sürükleyip döndürerek ajan kararlarını elle geçersiz kılabilmesi;
- Yeni bir `.blend` modelinin otomatik kategorize edilerek (geometri ölçümü + semantik türetim) kataloğa eklenmesi.

Kapsam dışı ve varsayımlar. Çalışma, geometriyi sıfırdan üretmek yerine katalogda hâlihazırda bulunan hazır varlıkları kullanır; dolayısıyla bir isteğin tam olarak karşılanabilmesi, uygun varlıkların katalogda mevcut olmasına bağlıdır. Fotogerçekçi görselleştirme (render), malzeme/ışık optimizasyonu ve fizik tabanlı çakışma çözümü bu sürümün kapsamı dışındadır. Ayrıca sistemin, masaüstü bir ortamda çalışan Blender 4.5+ ve Python 3.10+ altyapısı ile geçerli bir LLM sağlayıcı anahtarının (Gemini veya Claude) bulunduğu varsayılr. Sisteme özgü bilinen kısıtlar ve bunların etkileri ayrıca Bölüm 6.6 altında ele alınmaktadır.

2. LİTERATÜR TARAMASI VE İLGİLİ ÇALIŞMALAR

Bu bölümde, doğal dilden üç boyutlu (3B) içerik üretimi alanındaki temel çalışmalar incelenmekte ve bu çalışmanın söz konusu literatürdeki konumu belirlenmektedir. İncelenen kaynaklar, alanın genel gelişimini ele alan derlemeler [2,5-6] ile öncü üretken yöntemleri [3-4] kapsamaktadır.

2.1. Doğal Dilden 3B Üretimine Genel Bakış

Doğal dil betimlemelerinden 3B model üretimi, son yıllarda bilgisayar grafiği ve yapay zekânın kesişiminde hızla gelişen bir araştırma alanıdır. Lee ve arkadaşlarının derlemesinde [2] belirtildiği üzere, bu alandaki yöntemler son yıllarda Üretici Çekişmeli Ağlar (Generative Adversarial Networks, GAN) tabanlı yaklaşımlardan Difüzyon (Diffusion) modellerine doğru evrilmiştir. Li ve arkadaşlarının geniş kapsamlı derlemesi [5] ise alanı 3B veri temsilleri (voksel, çok-görünümlü görüntü, mesh, nokta bulutu ve sinirsel alanlar) ile çekirdek teknolojiler (nöral ışınım alanları — NeRF ve difüzyon modelleri) ekseninde sınıflandırarak, metinden 3B üretiminin sadakat, verimlilik, tutarlılık ve denetlenebilirlik boyutlarındaki gelişimini ortaya koymaktadır.

Bu derlemeler, alandaki temel gerilimi de görünür kılar: üretilen çıktıların görsel kalitesi artarken, bu çıktıların çoğu zaman doğrudan düzenlenebilir bir **mesh** yapısı yerine NeRF gibi sonradan düzenlenmesi zor temsillerde elde edildiği vurgulanmaktadır [5]. Bu çalışma, doğrudan düzenlenebilir, poligon (mesh) tabanlı varlıklar üzerinde çalışarak bu sınırlamadan ayrılmaktadır.

2.2. Üretken Yöntemler: GAN ve CLIP Tabanlı Yaklaşımlar

Doğal dilden 3B üretiminin ilk örnekleri, tekil nesnelerin geometrisini oluşturmaya odaklanmıştır. Fukamizu ve arkadaşları [3], doğal dil girdilerinden 3B model üretmek için koşullu bir GAN mimarisi kullanan öncü bir çalışma ortaya koymuş; düşük çözünürlüklü kaba bir şekli sonradan yüksek çözünürlüğe taşıyan iki aşamalı bir yöntem önermiştir. Ancak bu yaklaşımın karmaşık geometrilerde çözünürlük sınırlarına takıldığı görülmüştür.

Daha güncel yaklaşımlarda ise CLIP (Contrastive Language-Image Pre-training) modellerinin görsel-metinsel gömme uzayından yararlanılmaktadır. Örneğin CLIP-Sculptor çalışması [4], herhangi bir (metin, şekil) eğitim çiftine ihtiyaç duymadan, çok çözünürlüklü bir yaklaşımla sıfır-atış (zero-shot) yüksek sadakatli ve çeşitli şekiller üretebilmektedir. Bununla birlikte, bu tür üretken yöntemlerin çıktıları genellikle sonradan düzenlenebilir bir mesh yapısından ziyade düzenlenmesi zor formatlarda kalmaktadır [5]; bu da üretken yöntemleri, sahne kurgusu gibi denetlenebilirlik ve tekrarlanabilirlik gerektiren uygulamalardan ayırmaktadır.

2.3. Metinden 3B Sahne Üretimi ve Nesne–Mekân İlişkisi

Tekil nesnelerin ötesinde, nesnelerin birbiriyle ilişkisini içeren **sahne** üretimi daha karmaşık bir problemdir. Wen ve arkadaşları tarafından 2025 yılında yayımlanan kapsamlı derlemede [6], sahne üretiminin yalnızca nesne oluşturmak değil; aynı zamanda bu nesnelerin **mekânsal yerleşimini** ve

aralarındaki **anlamsal ilişkiyi** yönetmek olduğu vurgulanmaktadır. Bu derleme, sahne üretim yöntemlerini prosedürel üretim, nöral 3B tabanlı üretim, görüntü tabanlı ve video tabanlı üretim olmak üzere dört paradigmaya ayırmakta; sahne grafiği ve anlamsal yerleşim (semantic layout) gibi ara temsillerin, üretilen sahnedeki nesnelerin uzamsal düzenini denetlemek için kullanıldığını belirtmektedir.

Bu çalışmadaki "iç mekân mobilya tasarımı" kısıtı, doğrudan literatürdeki bu "nesne-mekân ilişkisi" problemine odaklanmaktadır: kullanıcının "masanın önünde sandalye olsun" gibi mekânsal ilişki ifade eden komutları, anlamsal bir katalog ve kural tabanlı bir yerleştirme algoritmasıyla işlenmektedir.

2.4. Bu Çalışmanın Konumu ve Farkı

Yukarıda incelenen mevcut "nöral işleme (neural rendering)" tabanlı sistemlerin aksine [5-6], bu çalışma prosedürel (kod tabanlı) bir modelleme/yerleştirme yöntemini seçerek üretilen sonuçların sonradan düzenlenebilir olmasını sağlamaktadır. Ayrıca literatürdeki ajan tabanlı çalışmalarda ajanların genellikle tekil görevlerde kullanıldığı görülürken; bu çalışmada büyük dil modeli destekli ajanların "Mekân Analizi", "Mobilya Seçimi", "Yerleşim" ve "Yürütme" olarak özelleşmiş alt kimliklere bölünmesinin, karmaşık istemlerin (prompt) yönetimini kolaylaştırdığı sonucuna varılmıştır.

Tablo 2.1, literatürdeki üretken/nöral metin→3B yöntemleri ile bu çalışmanın hazır-katalog ve kural tabanlı yaklaşımını karşılaştırmaktadır.

Tablo 2.1 İlgili çalışmalarla karşılaştırma.

Boyut	Üretken/nöral metin→3B [2-6]	Bu çalışma
Girdi	Doğal dil	Doğal dil
Varlık kaynağı	Üretilen geometri	Anlamsal v2 katalog (genişletilebilir)
Çıktı temsili	Çoğunlukla NeRF / düzenlenmesi zor	Düzenlenebilir mesh (poligon)
Yerleştirme	Öğrenilmiş/üretken	Kural + geometri + bağıl yönelim
Oda dışına taşma kontrolü	Belirsiz	İç hacim bbox kırpma (garanti)
Hedef yazılım	Çoğunlukla araştırma	Blender (gerçek üretim aracı)
LLM sağlayıcısı	Sabit	Sağlayıcıdan bağımsız (Gemini/Claude)

Bu çalışmanın ayırt edici yönleri; (i) düzenlenebilir mesh çıktısı veren prosedürel bir yerleştirme yaklaşımı, (ii) anlamsal olarak zenginleştirilmiş ve **otomatik kategorize ile genişleyebilen** bir katalog, (iii) mobilyaların oda/bina dışına taşmasını **garanti eden** iç hacim kırpma, ve (iv) hazır bir bina modelinin **belirli bir odasına** yerleştirme (Workflow B) yeteneğidir.

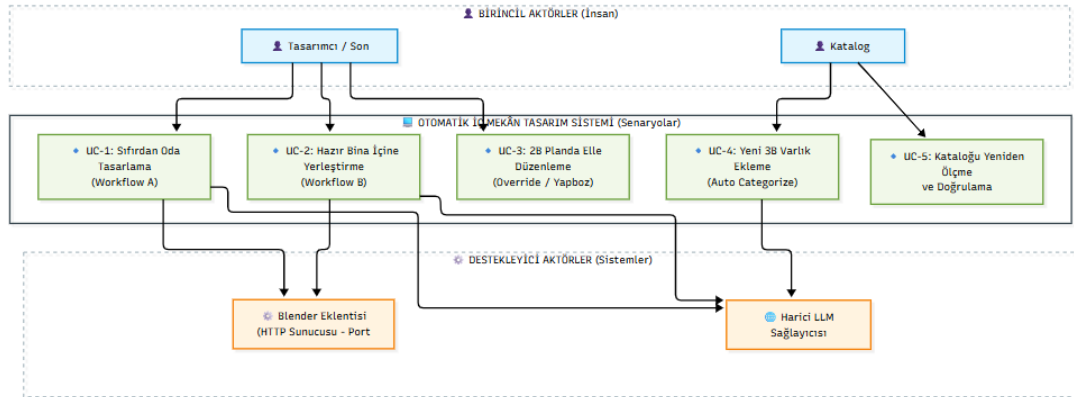
3. GEREKSİNİM ANALİZİ

3.1. Aktörler ve Kullanım Senaryoları

Sistemin başlıca aktörleri:

- **Tasarımcı / Son Kullanıcı:** Doğal dil ile istek girer, sonucu Blender'da görür, gerekirse 2B "Yapboz" arayüzüyle elle düzenler.
- **Katalog Yöneticisi:** `add_asset.py` ile kataloğa yeni 3B model ekler; düşük güvenli alanları onaylar.
- **Blender Eklentisi (sistem aktörü):** HTTP isteklerini alıp sahneyi inşa eder.
- **LLM Sağlayıcısı (dış sistem):** Gemini/Claude API'si.

Başlıca kullanım senaryoları: (UC-1) sıfırdan oda tasarlama, (UC-2) hazır bina içine yerleştirme, (UC-3) 2B planda elle düzenleme/override, (UC-4) yeni varlık ekleme, (UC-5) kataloğu yeniden ölçme/doğrulama.



Şekil 3.1 Sistemin kullanım senaryosu (use-case) diyagramı.

3.2. Fonksiyonel Gereksinimler

Sistemin, yukarıdaki kullanım senaryolarını karşılayabilmesi için yerine getirmesi gereken başlıca işlevsel gereksinimler (functional requirements, FR) Tablo 3.1'de listelenmiştir. Bu gereksinimler, doğal dil isteğinin çözümlenmesinden sahnenin Blender'da inşasına kadar uçtan uca akışı kapsar.

Tablo 3.1 Fonksiyonel gereksinimler.

No	Gereksinim
FR-1	Sistem, serbest doğal dil cümlesinden oda tipi, boyut, stil ve mobilya slotlarını çıkarmalıdır.
FR-2	Sistem, her slot için anlamsal katalogdan uygun varlığı (oda tipi/stil/uyumluluk filtreleriyle) seçmelidir.
FR-3	Sistem, mobilyaların konum ve dönüş açısını oda sınırları ve bağlı ilişkiler (önünde/yanında) gözetilerek hesaplamalıdır.
FR-4	Sistem, hesaplanan sahneyi Blender'da otomatik inşa etmelidir (oda + mobilya içe aktarma).
FR-5	Sistem, hazır bir bina modelinin belirtilen odasına yerleştirme yapabilmeli ve mobilyayı iç hacme kırpmalıdır (Workflow B).
FR-6	Sistem, kullanıcının 2B kuşbakışı planda mobilyaları sürükleyip döndürerek ajan kararlarını geçersiz kılmasına izin vermelidir.
FR-7	Sistem, yeni bir .blend modelini otomatik kategorize ederek (geometri ölçümü + semantik türetim) kataloğa ekleyebilmelidir.
FR-8	Sistem, en az iki LLM sağlayıcısı (Gemini/Claude) arasında seçim yapabilmelidir.

3.3. Fonksiyonel Olmayan Gereksinimler

İşlevsel gereksinimlerin yanı sıra, sistemin hangi kalite niteliklerini taşıması gerektiğini tanımlayan fonksiyonel olmayan gereksinimler (non-functional requirements, NFR) Tablo 3.2'de verilmiştir. Bu nitelikler, izleyen bölümlerdeki mimari ve gerçekleştirim kararlarını doğrudan biçimlendirmiştir.

Tablo 3.2 Fonksiyonel olmayan gereksinimler.

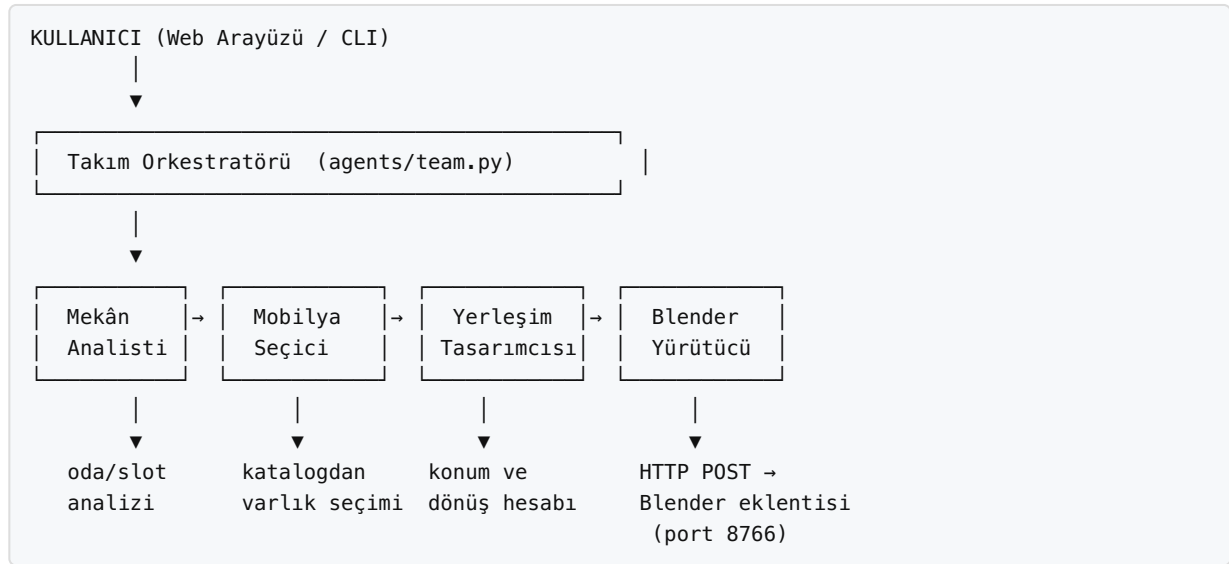
No	Gereksinim	Açıklama
NFR-1	Modülerlik	Her ajan/araç bağımsız geliştirilip değiştirilebilir olmalıdır.
NFR-2	Genişletilebilirlik	Yeni varlık ve oda tipleri kod değişikliği olmadan eklenebilmelidir.
NFR-3	Sağlayıcı bağımsızlığı	Sistem tek bir LLM sağlayıcısına bağımlı olmamalıdır.
NFR-4	Tekrarlanabilirlik	Aynı istek, makul ölçüde aynı sonucu üretmelidir (temperature=0).
NFR-5	Güvenilirlik	Mobilya, oda iç boyutlarına sığıldığı sürece oda/bina dışına taşmamalı; konumu iç hacme kırpılmalıdır.
NFR-6	Kullanılabilirlik	Kullanıcı 3B yazılım bilgisi olmadan tek cümleyle tasarım yapabilmelidir.
NFR-7	Taşınabilirlik	Sistem masaüstü ortamlarda (Blender 4.5+, Python 3.10+) çalışabilmelidir.

4. SİSTEM MİMARİSİ VE TASARIM

4.1. Genel Mimari ve Katmanlı İşlem Hattı

Sistem, kullanıcı arayüzünden başlayıp Blender sahnesinde sonlanan dört katmanlı bir işlem hattı üzerine kuruludur. Kullanıcı isteği, ister komut satırı (CLI) ister web arayüzü üzerinden gelsin, bir takım orkestratörü (`agents/team.py`) tarafından sırasıyla dört uzman ajana yönlendirilir. Her ajan kendi uzmanlık alanındaki çıktısını üretir ve bu çıktıyı bir sonraki ajana aktarır. Son ajan, hesaplanan sahneyi HTTP üzerinden Blender içindeki eklentiye göndererek fiziksel olarak inşa eder.

İşlem hattının yüksek seviyeli akışı Şekil 4.1'de gösterilmiştir.



Şekil 4.1 Sistemin yüksek seviyeli işlem hattı (pipeline) akışı.

İki giriş noktası bulunur: web arayüzü ve API için `run_team.py --serve` (FastAPI, port **8080**) [8], doğrudan komut satırı kullanımı için ise `run_team.py "<istek>"`. Blender tarafı, eklenti içinde çalışan ayrı bir HTTP sunucusu (port **8766**) üzerinden dinler.

4.2. v2 Katalog Şeması ve Semantik Katman

Sistemin merkezinde, tüm 3B varlıkların metadatasını barındıran `Catalog.json` dosyası ve bu dosyayı okuyup yardımcı sorgular sunan `mcp_server/catalog.py` kütüphanesi bulunur. Katalog, sürüm **2.0.0** şemasındadır ve klasik sınıflandırma (kategori/alt-kategori/stil) bilgisinin

yanına, ajanların akıllı seçim yapabilmesi için bir **anlamsal (semantik) katman** ile **geometri** ve **yerleştirme** bilgisi ekler.

Tablo 4.1 v2 katalog şeması alanları.

Katman	Alan	Açıklama
Taksonomi	id, category, subcategory, style, tags	Temel sınıflandırma
Semantik	semantic_tags	Tema/niyet etiketleri (ör. office, work, rest)
Semantik	room_types	Varlığın hangi oda tiplerine uygun olduğu
Semantik	is_container	Varlık bir bina/ev mi?
Semantik	scale_class	human architectural decor
Semantik	compatible_with	Birlikte kullanılan eş varlık alt-kategorileri
Geometri	footprint	Taban ayak izi + ön/yan açıklık payları
Geometri	dimensions_m, poly_count	Boyut ve poligon sayısı
Yerleştirme	placement.forward_axis	Modelin ön yön eksenini (+Y/-Y/+X/-X)
Yerleştirme	placement.rules, placement.anchor	Yerleştirme kuralları ve dayanak noktası
Yerleştirme	placement.confidence	Otomatik kategorize güven değeri

Bina/ev türü varlıklar (`is_container: true`) ek olarak `container_meta` (iç hacim sınırlayıcı kutusu, zemin kotu, giriş noktası) ve `rooms` (bina içindeki her odanın `origin_offset_m` öteleme vektörü, boyutları ve `interior_bbox_local` iç hacim kutusu) alanlarını taşır. Bu yapı, mobilyaların yalnızca bir oda değil, çok katlı bir binanın belirli bir odası içine yerleştirilmesini mümkün kılar.

`catalog.py` kütüphanesi şu yardımcı işlevleri sunar: `get_all_assets`, `get_asset_by_id`, `get_forward_axis`, `get_footprint`, `is_container`, `get_interior_bbox`, `get_room_interior_bbox`, çok ölçütlü filtreleme yapan `search_assets` ve eş varlıkları getiren `list_companions`.

Örnek bir varlık metadatası (kısaltılmış):

```
{
  "id": "metal_office_desk_4k",
  "category": "tables", "subcategory": "desk",
  "style": ["industrial"], "tags": ["metal", "office"],
  "semantic_tags": ["office", "work", "industrial"],
  "room_types": ["office", "home_office", "study"],
  "is_container": false, "scale_class": "human",
  "compatible_with": ["office_chair", "armchair"],
  "footprint": { "width_m": 2.0, "depth_m": 0.947, "height_m": 0.788,
    "clearance_front_m": 0.8, "clearance_sides_m": 0.1 },
  "placement": { "rules": ["floor", "wall_optional", "center_ok"],
    "anchor": "floor_center", "forward_axis": "-Y",
    "facing_correction_z": 0, "confidence": 1.0 },
  "dimensions_m": { "width": 2.0, "depth": 0.947, "height": 0.788 },
  "poly_count": 3557
}
```

Şekil 4.2 v2 katalog şemasının katmanları ve örnek varlık metadatası.

4.3. Çok-Ajanlı Tasarım Takımı

Takım, `agno` çok-ajanlı orkestrasyon çerçevesi kullanılarak kurulmuştur [9]. `team.py` içindeki `_get_model` işlevi kullanılan büyük dil modelini belirler: varsayılan olarak **gemini-2.5-flash** [10], isteğe bağlı olarak **claude-sonnet-4-6** [11]. Dört ajan, çıktısı bir sonrakinin girdisi olacak şekilde sıralı (sequential) çalışır.

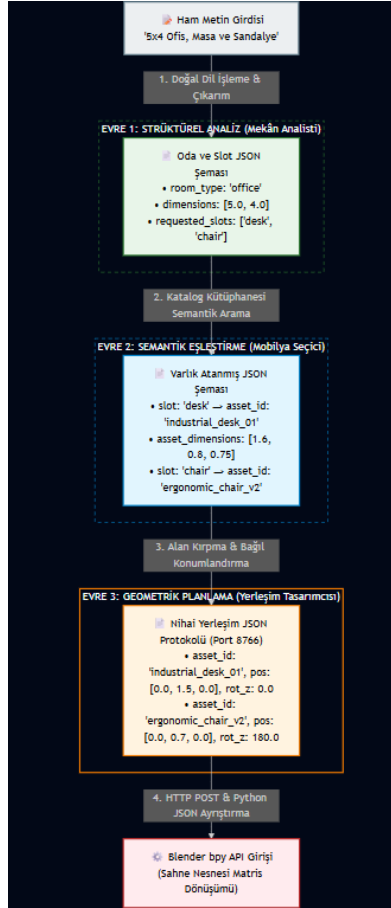
Tablo 4.2 Ajanlar: roller, modeller ve kullandıkları araçlar.

Ajan	Rolü	Başlıca Araçları
Mekân Analisti (<code>space_analyst.py</code>)	İsteği çözümler; oda tipi/boyut/stil ve mobilya slot listesini üretir; Workflow A/B kararını verir	<code>get_room_preset</code> , <code>search_catalog</code> , <code>list_house_assets</code> , <code>list_house_rooms</code> , <code>list_companions</code>
Mobilya Seçici (<code>furniture_selector.py</code>)	Her slot için katalogdan en uygun varlığı seçer; eş varlıkları ekler	<code>search_catalog</code> , <code>get_asset_details</code> , <code>list_companions</code>
Yerleşim Tasarımcısı (<code>layout_designer.py</code>)	Her mobilyanın konum ve dönüşünü hesaplar	<code>calculate_furniture_layout</code>
Blender Yürütücü (<code>blender_executor.py</code>)	Oda ve mobilyaları Blender'da inşa eder	<code>check_blender_connection</code> , <code>create_room_in_blender</code> , <code>import_assets_to_blender</code>

Takım iki ana akışı yönetir. **Workflow A** (sıfırdan oda): Mekân Analisti istekte bir bina modeli tespit etmezse, Blender Yürütücü önce `create_room_in_blender` ile zemin ve dört duvarı oluşturur, ardından mobilyaları içe aktarır. **Workflow B** (hazır ev içinde): İstekte "ev", "bina", "içinde" gibi ifadeler ya da bir bina kimliği geçerse; Mekân Analisti `list_house_assets` ve

`list_house_rooms` ile ilgili binayı ve odayı bulur, Yerleşim Tasarımcısı ilgili `container_id` ve `room_id` ile çalışarak mobilyaları odanın iç hacmine kırpar, Blender Yürütücü ise önce binayı `[0,0,0]` konumunda içe aktarır sonra mobilyaları yerleştirir.

Ajanlar arası sözleşmenin kritik bir yönü, Yerleşim Tasarımcısı ile Blender Yürütücü arasındaki **veri bütünlüğüdür**: Yerleşim Tasarımcısı'na, `calculate_furniture_layout` aracının ürettiği JSON çıktısını hiçbir alanını değiştirmeden, özetlemeden ve markdown kod bloğuna sarmadan birebir aktarması talimatı verilmiştir; çünkü Blender Yürütücü bu çıktıyı doğrudan JSON olarak ayrıştırmaktadır.



Şekil 4.3 Çok-ajanlı takımın etkileşim/akış diyagramı.

4.4. Yerleştirme ve Geometri Mantığı

Sistemin en kritik algoritmik bileşeni, `agents/tools/layout_tools.py` içindeki `calculate_furniture_layout` işlevidir. Bu işlev üç temel görevi yerine getirir:

(1) İç hacim sınırlayıcı kutu (interior bbox) kırpma. Eğer bir `container_id` ve `room_id` verilmişse, ilgili binanın oda iç hacmi kataloğdan `get_room_interior_bbox` ile alınır. Bina dünya orijininde (`[0,0,0]`), dönüşsüz) içe aktarıldığından, yerel iç hacim kutusu doğrudan dünya kutusu olarak kullanılabilir. Workflow A'da ise iç hacim kutusu, oda boyutlarından duvar kalınlığı ve pay (margin) düşülerek sentezlenir. Hesaplanan her mobilya konumu bu kutunun sınırlarına çekilir; böylece **mobilya hiçbir koşulda oda/bina dışına taşmaz**.

(2) Kırpmanın bağıl yerleşimlere yansıtılması. Bir masa, iç hacim kutusuna kırıldığında konumu değişebilir. Ona referans veren bir sandalyenin ("masanın önünde") doğru konumlanması için, masanın *kırılmış* konumunun yerleştirme kayıt yapısına (`placed[]`) yazılması gerekir. Bu sayede sonraki bağıl hesaplamalar kırılmış konumu görür ve nesneler arası görelî geometri korunur. (Bu, ilk sürümde gözden kaçıp düzeltilen kritik bir hata olmuştur.)

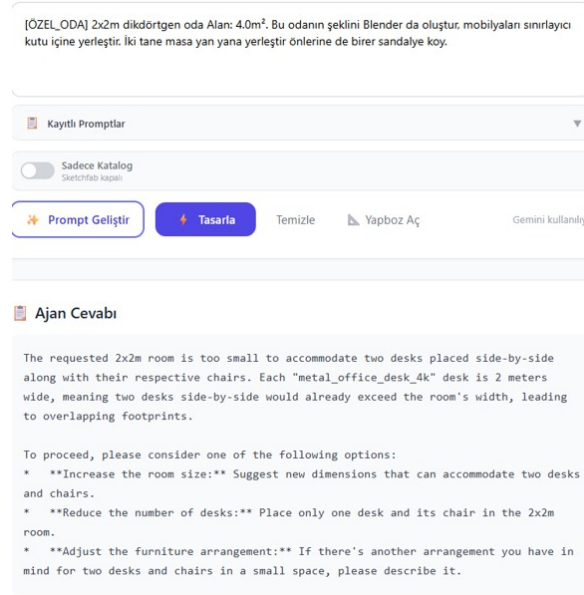
(3) Bağıl yönelim (relative facing). "Önünde" (`in_front_of:`) veya "masa kenarı" (`table_side:`) gibi yerleştirmelerde, ilgili nesnenin referans nesneye dönük olması için gerekli dönüş açısı `interior_design_tools.py` içindeki yönelim işlevleriyle hesaplanır. Buna varlığın katalogdaki `forward_axis` değerinden türetilen `facing_correction_z` düzeltilmesi de eklenir; çünkü farklı modeller farklı ön yön eksenlerine sahip olabilir (mevcut katalogdaki varlıklar hem `+Y` hem `-Y` ön eksenlidir; bu nedenle `facing_correction_z` her varlık için ayrı belirlenir).

Algoritma, yerleştirmeleri öncelik sırasına göre işler: önce odaklanma noktaları (masa, yemek masası, sehpa), sonra bunlara bağıl yerleşimler, en son duvar/köşe/merkez yerleşimleri. Ayrıca duvara yaslanan mobilyalarda çıkışma tespiti için genişlik ve derinlik değerleri dönüşe göre yer değiştirilir (rotation-aware footprint).

Tablo 4.3 Desteklenen yerleştirme (placement) tipleri.

Tür	Değerler
Duvar	<code>north_wall</code> , <code>south_wall</code> , <code>east_wall</code> , <code>west_wall</code>
Köşe	<code>corner_nw</code> , <code>corner_ne</code> , <code>corner_sw</code> , <code>corner_se</code>
Merkez	<code>center</code>
Bağıl	<code>in_front_of:<slot></code> , <code>beside:<slot>:left/right</code> , <code>table_side:n/s/e/w</code>

Yukarıda açıklanan iç hacim kırpma adımının bir mobilya yerleşimi üzerindeki etkisi Şekil 4.4'te görselleştirilmiştir: hesaplanan konum, iç hacim kutusunun sınırlarına çekilerek mobilyanın oda/bina dışına taşması engellenir.



Şekil 4.4 İç hacim sınırlayıcı kutu (interior bbox) kırpma görselleştirmesi.

4.5. Blender Eklentisi ve HTTP Köprüsü

Blender tarafında, sahneyi gerçek zamanlı inşa eden bir Python eklentisi (`blender_addon/`) çalışır [12]. Eklenti kurulduktan sonra Blender'ın yan panelinde (N tuşu) "Asset Library" sekmesi belirir; buradaki "Start Server" düğmesi, port **8766** üzerinde dinleyen, Blender'ın ana iş parçacığını kilitlemeyen (non-blocking) bir HTTP sunucusu başlatır. Dış dünyadan gelen JSON istekleri bir kuyruğa alınır, eklentinin modal zamanlayıcısı tarafından işlenir ve yanıt geri döndürülür.

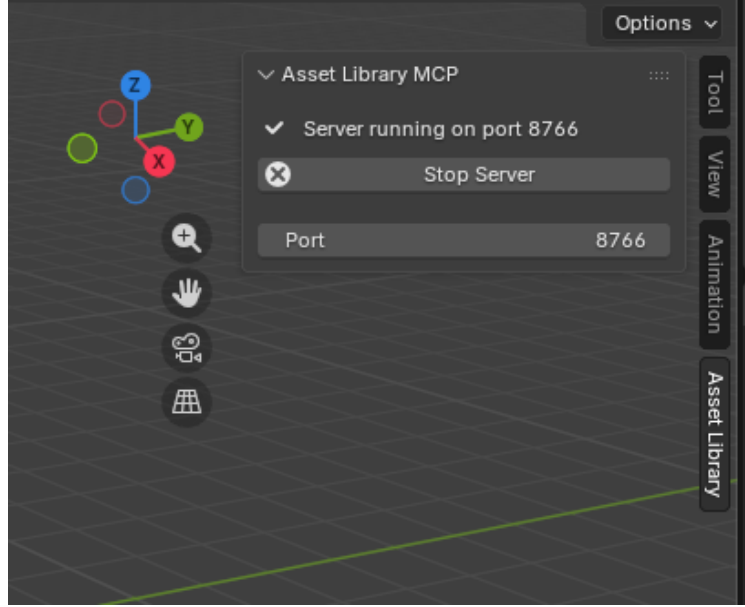
Eklenti, bir eylem-işleyici (action-handler) kayıt defteri üzerinden çalışır:

Tablo 4.4 Blender eklentisi HTTP eylem (handler) listesi.

Eylem	İşlev
<code>import_blend_asset</code>	Bir <code>.blend</code> dosyasını içe aktarır; konum, dönüş ve ölçeği uygular
<code>create_room</code>	Zemin + dört duvardan oluşan oda oluşturur
<code>list_scene_objects</code>	Sahnedeki nesneleri dünya konumu/boyutlarıyla listeler
<code>test_asset_facing</code>	Bir varlığı orijinde dönüşsüz içe aktarır (ön yön tespiti için)
<code>get_3d_cursor</code>	3B imlecin konumunu döndürür (oda orijini ölçümü için)
<code>delete_objects</code>	İsim listesine göre nesneleri siler

Varlık içe aktarımı (`import_blend_asset`); içe aktarmadan önceki ve sonraki nesne kümelerini karşılaştırarak yeni eklenen nesneleri tespit eder, bunları sahne koleksiyonuna bağlar, kök (parent'sız) nesneleri bularak konum/dönüş/ölçek dönüşümlerini yalnızca bu köklere uygular. Dönüş açıları dereceden radyana çevrilerek `rotation_euler` alanına işlenir.

`get_3d_cursor` ve `test_asset_facing` eylemleri, sistemin **kalibrasyon** ihtiyaçlarına hizmet eder: bir bina modeli içindeki oda orijinlerini ölçmek için kullanıcı Blender'da 3B imleci ilgili odaya yerleştirip koordinatı okur; bir varlığın ön yönünü belirlemek için ise varlık dönüşsüz içe aktarılıp görsel olarak incelenir.



Şekil 4.5 Blender eklentisi N-paneli ve "Start Server" düğmesi.

4.6. Otomatik Kategorize (`auto_categorize`) İşlem Hattı

Yeni 3B modellerin kataloğa tutarlı biçimde eklenmesi, sistemin sürdürülebilirliği için kritiktir. `auto_categorize.py`, bir `.blend` dosyasını alıp v2 katalog girdisi üretmek için beş aşamalı hibrit bir akış uygular:

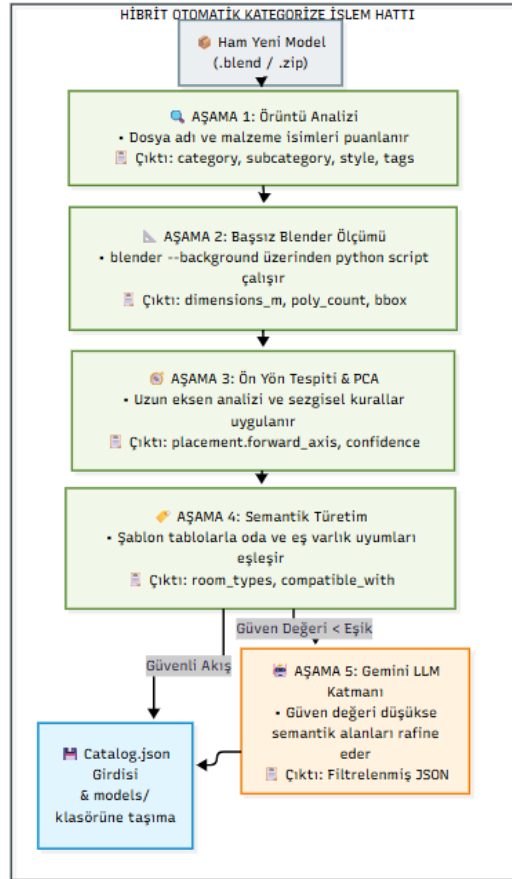
1. **Dosya adı + malzeme örüntü analizi:** Dosya adı ve malzeme isimleri normalize edilip anahtar kelime puanlamasıyla kategori, alt-kategori, stil ve etiketler tahmin edilir. Her tahmine bir güven (confidence) değeri atanır.
2. **Blender ile başsız (headless) ölçüm:** `blender --background <model> --python <ölçüm betiği>` çağrısıyla; sınırlayıcı kutu, genişlik/derinlik/yükseklik, poligon sayısı, malzeme yuvaları ve XY düzleminde temel bileşen analizi (PCA) açısı ölçülür [13].
3. **Ön yön (forward axis) tespiti:** PCA uzun eksenini ve kategori bazlı sezgisel (heuristic) kurallarla modelin ön yönü tahmin edilir; simetrik nesnelerde güven düşük tutulur.
4. **Semantik katman türetimi:** Alt-kategoriden oda tipleri (`SUBCAT_TO_ROOMS`) ve eş varlıklar (`SUBCAT_COMPATIBLE`) eşleştirmeleriyle `room_types`, `compatible_with` ve `semantic_tags` üretilir.

5. **Dil modeli ile zenginleştirme (opsiyonel):** Toplam güven eşiğinin altındaysa ve `GOOGLE_API_KEY` tanımlıysa, Gemini modeli `temperature=0` ve JSON yanıt biçiminde çağrılarak semantik alanlar rafine edilir; çıktı izinli kelime dağarcığına göre filtrelenir [10].

Tablo 4.5 Otomatik kategorize alt-kategori eşleştirmeleri (örnek).

Alt-kategori	room_types	compatible_with
desk	office, home_office, study	office_chair, table_lamp, bookshelf
sofa	living_room, lobby	coffee_table, side_table, floor_lamp, rug
dining_table	dining_room, kitchen	dining_chair

Bu akış, `add_asset.py` aracı içine entegre edilmiştir; düşük güvenli alanlar kullanıcıya onay için gösterilir, onaylanan varlık `models/<kategori>/<id>/` altına taşınır ve `Catalog.json`'a yazılır. `recategorize_existing.py` ise mevcut kataloğu yeniden ölçüp farkları yan yana (diff) gösteren doğrulama aracıdır.

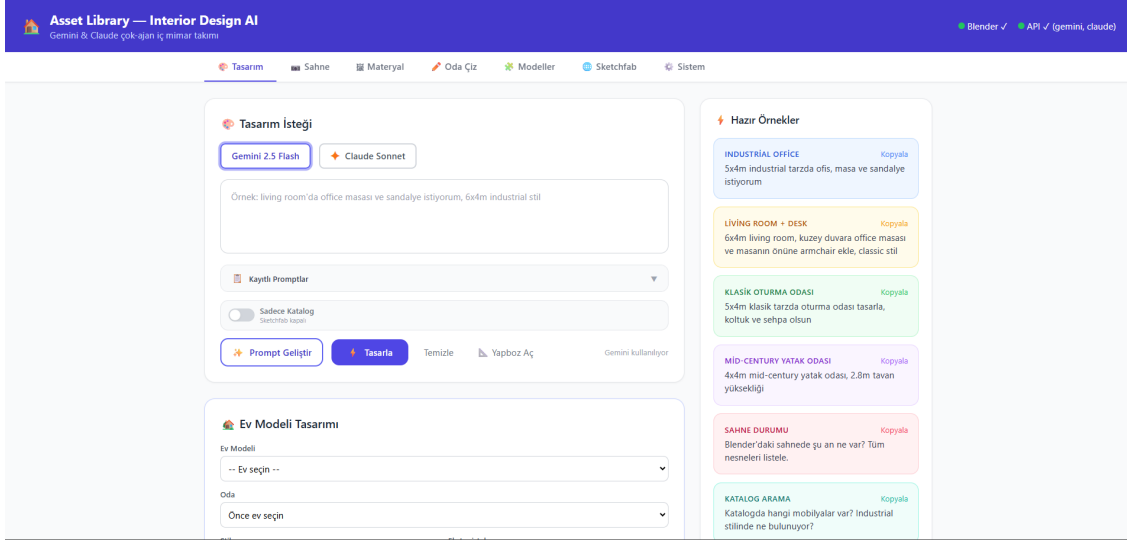


Şekil 4.6 auto_categorize hibrit kategorize işlem hattı.

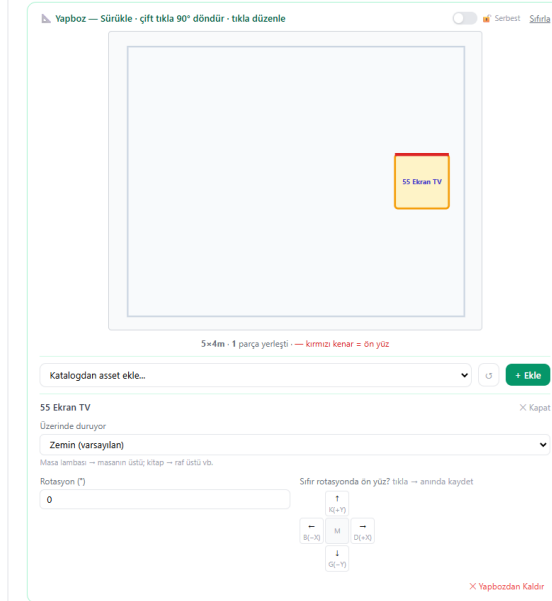
4.7. Web Tabanlı Kullanıcı Arayüzü

`prompt_builder.html`, kullanıcıların doğal dil isteklerini girip sonuçları izleyebildiği tarayıcı tabanlı arayüzdür (Tailwind CSS ile biçimlendirilmiş, API'ye asenkron `fetch` çağrıları yapan tek sayfalık uygulama). Arayüz sekmelere ayrılmıştır: **Tasarım, Sahne, Materyal** ve **Oda Çiz**.

"Tasarım" sekmesi şu yetenekleri sunar: kullanılacak dil modelinin seçimi (Gemini / Claude), serbest metin giriş alanı, isteği yapılandırılmış slot listesine dönüştüren **Prompt Geliştir** özelliği ve mobilyaların 2B kuşbakışı plan üzerinde sürüklenerek, çift tıklı 90° döndürülerek ve yön pusulasıyla düzenlenebildiği **Yapboz** bileşeni. Kullanıcı bu 2B düzenlemeyle ajanların otomatik kararlarını elle geçersiz kılabilir (override). Ayrıca ayrı bir "Ev Modeli Tasarımı" bölümünde bina ve oda seçici, stil seçici ve Blender'ın 3B imlecini okuyup oda orijini olarak kaydeden bir araç bulunur.



Şekil 4.7 Web arayüzü — "Tasarım" sekmesi.



Şekil 4.8 Web arayüzü — "Yapboz" 2B yerleşim düzenleyici.

4.8. Teknoloji Yığını

Sistemin gerçekleştirilmesinde kullanılan başlıca teknolojiler, kütüphaneler ve bunların projedeki rolleri Tablo 4.6'da özetlenmiştir. Çekirdek mantık Python ile yazılmış olup ajan orkestrasyonu, katalog yönetimi, Blender entegrasyonu ve web sunucusu birbirinden bağımsız bileşenler hâlinde ele alınmıştır; web arayüzü ise HTML/CSS/JavaScript ile gerçekleştirilmiştir.

Tablo 4.6 Teknoloji yığını ve bağımlılıklar.

Bileşen	Teknoloji / Kütüphane
Programlama dili	Python ≥ 3.10
Çok-ajanlı orkestrasyon	agno [9]
Araç sunucusu	FastMCP (mcp [cli]) — Model Context Protocol [14]
Web/API sunucusu	fastapi [8] + uvicorn (ASGI) [15]
HTTP istemcisi	httpx [16]
Veri doğrulama	pydantic [17]
LLM sağlayıcıları	google-genai (Gemini) [10], anthropic (Claude) [11]
Yapılandırma	python-dotenv (.env)
3B ortam	Blender 4.5+ (Python eklentisi, bpy) [12]
Arayüz	HTML + Tailwind CSS + JavaScript (SVG)

Yapılandırma `asset-library-mcp/.env` dosyasından okunur ve şu anahtarları içerir: `GOOGLE_API_KEY`, `ANTHROPIC_API_KEY` ve katalog dizinini belirten `ASSET_LIBRARY_DIR`.

5. GERÇEKLEŞTİRİM (UYGULAMA) DETAYLARI

5.1. Proje Dizin Yapısı

Projenin dizin yapısı, sorumlulukların net biçimde ayrıldığı modüler bir düzendedir. Dizinlerin ve başlıca dosyaların yorumlarıyla birlikte tam ağaç görünümü Şekil 5.1'de verilmiştir.

```
asset_library/
├─ Catalog.json           # v2 şema: tüm asset metadata
├─ auto_categorize.py     # Blender headless + PCA + heuristic + Gemini hibrit
├─ add_asset.py           # Yeni asset ekleme CLI'ı (auto_categorize entegre)
├─ recategorize_existing.py # Mevcut katalog'u yeniden ölçer (side-by-side diff)
├─ schema/catalog_schema.json # v2 JSON Schema referansı
├─ models/                # .blend dosyaları + textures
│   ├── seating/
│   ├── tables/
│   ├── kitchen/
│   └─ architecture/
├─ zip/                   # Henüz çıkartılmamış asset zipleri
├─ asset-library-mcp/     # Python paketi
│   ├── .env              # GOOGLE_API_KEY, ANTHROPIC_API_KEY, ASSET_LIBRARY_DIR
│   ├── pyproject.toml
│   ├── run_team.py       # API server entry point (port 8080)
│   ├── prompt_builder.html # Tarayıcı UI'ı
│   ├── make_addon_zip.py # Blender addon zip'ler
│   ├── asset_library_mcp_addon.zip # Blender addon (kurulu olmalı)
│   └─ mcp_server/        # FastMCP sunucusu + catalog kütüphanesi
│       ├── catalog.py    # Catalog.json okuma + v2 helper'lar
│       ├── server.py
│       ├── blender_client.py
│       └─ tools/
│           ├── catalog_tools.py
│           ├── catalog_management_tools.py
│           ├── import_tools.py
│           └─ interior_design_tools.py # calculate_position, clamp, relative_facing
├─ agents/                # agno multi-agent takımı
│   ├── team.py           # Team coordinator (workflow A/B)
│   ├── space_analyst.py
│   ├── furniture_selector.py
│   ├── layout_designer.py
│   ├── blender_executor.py
│   ├── api.py
│   └─ tools/
│       ├── catalog_tools.py # agno için catalog wrapper
│       ├── layout_tools.py  # calculate_furniture_layout (container-aware)
│       └─ blender_tools.py  # HTTP -> Blender addon (port 8766)
├─ blender_addon/         # Blender içi HTTP server
│   ├── __init__.py       # Addon entry
│   ├── server/http_server.py # Port 8766 listener
│   ├── operators/server_operator.py # N panel "Start Server" butonu
│   └─ handlers/
│       ├── blend_import.py # import_blend_asset (location, rotation, scale uygula)
│       └─ room.py          # create_room, list_scene_objects, test_asset_facing,
```

Şekil 5.1 Proje dizin yapısı (ağaç görünümü).

5.2. Kurulum ve Çalıştırma

Sistem aşağıdaki adımlarla kurulur ve çalıştırılır:

```
# 1) Bağımlılıklar
cd asset-library-mcp
pip install -e .

# 2) .env doldur
#   GOOGLE_API_KEY=...
#   ANTHROPIC_API_KEY=...
#   ASSET_LIBRARY_DIR=<asset_library mutlak yolu>

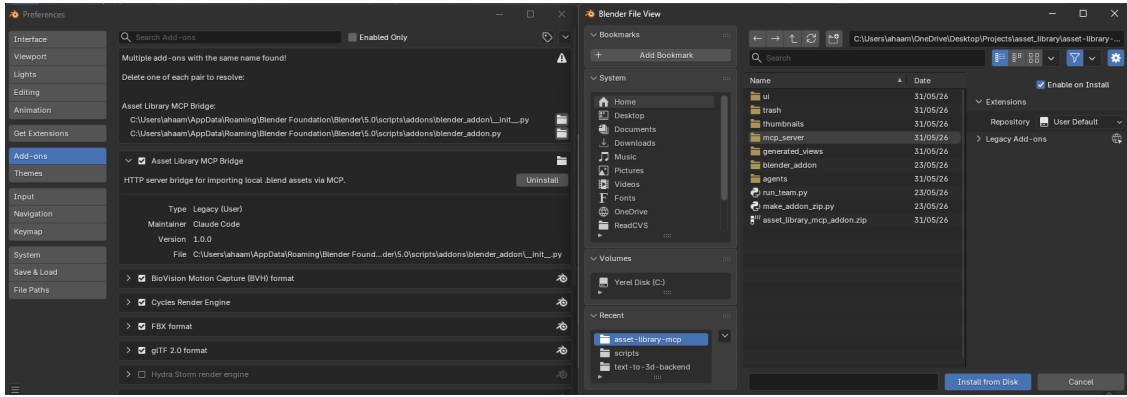
# 3) Blender eklentisini kur (bir kez)
#   Blender > Edit > Preferences > Add-ons > Install
#   asset-library-mcp/asset_library_mcp_addon.zip seç, aktif et
#   N paneli > "Asset Library" > "Start Server" (port 8766)

# 4) API + arayüzü çalıştır
python run_team.py --serve
#   → http://localhost:8080/prompt_builder.html

# 5) (Opsiyonel) Komut satırından tek istek
python run_team.py "5x4 ofise endüstriyel masa ve sandalye koy"

# 6) Yeni varlık ekle
cd ..
python add_asset.py model.zip # interaktif (düşük güveni sorar)
python add_asset.py model.blend --quick # otomatik

# 7) Kataloğu yeniden ölç / doğrula
python recategorize_existing.py --dry-run # sadece farkı göster
python recategorize_existing.py --auto-accept # değişiklikleri uygula
```



Şekil 5.2 Kurulum: Blender eklentisinin yüklenmesi.

5.3. Uçtan Uca Bir İsteğin Akışı (Sequence)

Bir kullanıcı isteğinin uçtan uca işlenmesi şu sırayı izler:

```
Kullanıcı → Web UI/CLI : "5x4 ofise endüstriyel masa ve sandalye koy"
Web UI/CLI → team.py : run(istek, model)
team.py → Mekân Analisti: cümleyi çözümü
Mekân Analisti → Katalog: search_catalog / get_room_preset
Mekân Analisti → team.py: {oda, slotlar, workflow}
team.py → Mobilya Seçici: slotları doldur
Mobilya Seçici → Katalog: search_catalog / get_asset_details
Mobilya Seçici → team.py: seçilen varlıklar (JSON)
team.py → Yerleşim Tas. : calculate_furniture_layout(...)
Yerleşim Tas. → team.py : konum/dönüş JSON (kırpılmış)
team.py → Blender Yürüt. : create_room / import_assets
Blender Yürüt. → Eklenti : HTTP POST (port 8766)
Eklenti → Blender Sahnesi: oda + mobilya inşa
Eklenti → Kullanıcı : sonuç sahnesi + durum
```

Şekil 5.3 Uçtan uca bir tasarım isteğinin sıra (sequence) diyagramı.

5.4. Kritik Kod Bileşenleri

Sistemin doğru çalışması için en kritik üç kod mantığı şunlardır:

1. **calculate_furniture_layout** (agents/tools/layout_tools.py): `container_id` + `room_id` verildiğinde `get_room_interior_bbox` ile iç hacmi alır, her mobilya konumunu bu kutuya kırpar, kırpılmış konumu `placed[]` 'a yazar ve `in_front_of:` / `table_side:` için bağıl yönelimi hesaplar.
2. **relative_facing_rot_z** ve **clamp_to_interior_bbox** (mcp_server/tools/interior_design_tools.py): Sırasıyla bir nesnenin referansa dönük olması için gereken `rot_z` açısını ve bir konumun iç hacme kırpılmasını hesaplar; footprint-duyarlı konumlandırma sağlar.
3. **import_blend_asset** (blender_addon/handlers/blend_import.py): İçerik aktarma öncesi/sonrası nesne kümelerini karşılaştırıp yeni kökleri bulur ve konum/dönüş/ölçek dönüşümlerini yalnızca köklere uygular; dereceyi radyana çevirir.

6. BULGULAR VE TARTIŞMA

6.1. Test Yöntemi

Sistem, birim testleri yerine **uçtan uca (end-to-end) manuel doğrulama** ile test edilmiştir: belirli doğal dil istekleri web arayüzü/CLI üzerinden gönderilmiş, ajan çıktıları (slot listesi, seçilen varlıklar, konum/dönüş JSON'u) ve nihai Blender sahnesi gözlemlenmiştir. Değerlendirme ölçütleri: (i) doğru oda/boyut çıkarımı, (ii) uygun varlık seçimi, (iii) mobilyaların oda içinde kalması (taşmama), (iv) bağlı yönelim doğruluğu (sandalyenin masaya dönük olması).

6.2. Workflow A — Sıfırdan Oda Kurulumu

Workflow A, herhangi bir hazır bina modeli olmadan, oda geometrisinin sistem tarafından oluşturulduğu akıştır. Örnek bir istek ve sistemin ürettiği yanıt zinciri:

```
Kullanıcı: "5x4 ofise endüstriyel masa ve sandalye koy"

Mekân Analisti → bina tespit edilmedi → Workflow A
  Oda: office, 5m × 4m × 2.7m, stil: industrial
  Slotlar:
    - slot: desk, placement: north_wall
    - slot: chair, placement: in_front_of:desk

Mobilya Seçici → search_catalog(subcategory=desk, room_type=office, style=industrial)
  Seçilenler: metal_office_desk_4k, mid_century_lounge_chair_4k

Yerleşim Tasarımcısı → calculate_furniture_layout(5, 4, 2.7, ..., container_id="", room_id="")
  desk → location ≈ [0, 1.18, 0], rot_z (forward_axis düzeltmeli)
  chair → location önünde, masaya dönük

Blender Yürütücü → create_room_in_blender(5, 4, 2.7)
  → import_assets_to_blender(<yerleşim JSON>)
```


Bu akışta masa kuzey duvarına yaslanır, sandalye masanın önüne ve masaya dönük olacak şekilde otomatik döndürülür. Oda sınırları, sentezlenen iç hacim kutusuyla zorlanır; böylece mobilyalar duvarların içinden taşmaz.



Şekil 6.1 Workflow A çıktısı — sıfırdan oluşturulan oda ve mobilyalar.

6.3. Workflow B — Hazır Ev Modeli İçinde Yerleştirme

Workflow B, mobilyaların önceden hazırlanmış bir bina/ev modelinin belirli bir odasına yerleştirildiği, sistemin en ayırt edici yeteneğidir. Mekân Analisti istekteki "ev", "bina", "üst kat" gibi ifadelerden Workflow B'ye geçer; ilgili binayı ve odayı katalogdan bulur; Yerleşim Tasarımcısı odanın `interior_bbox_local` iç hacmiyle kırpmayı yapar; Blender Yürütücü önce binayı `[0,0,0]` konumunda, dönuşsüz içe aktarır, ardından mobilyaları yerleştirir. Bina dünya orijininde

olduğundan, odanın yerel iç hacim kutusu doğrudan dünya kutusu olur ve mobilyalar binanın ilgili odasının içinde kalır.



Şekil 6.2 Workflow B çıktısı — bina modeli içinde yerleştirilmiş mobilyalar.

6.4. Asset Ekleme ve Otomatik Kategorize Sonuçları

Mevcut katalog, v2 şemasına taşınmış on varlık içermektedir (Tablo 6.1). Varlıklar; oturma (sofa, koltuk, ofis sandalyesi), masa (çalışma masası), depolama (gardırop, TV ünitesi) ve mutfak (buzdolabı) gibi farklı kategorilere yayılmıştır ve hem +Y hem -Y ön yön eksenli modeller içermektedir.

Tablo 6.1 Mevcut katalog varlıkları.

#	ID	Kategori / Alt-kategori	Ön yön (forward_axis)
1	sofa_leather_black_01	seating / sofa	-Y
2	metal_office_desk_4k	tables / desk	-Y
3	mid_century_lounge_chair_4k	seating / armchair	-Y
4	artichoke_red	seating / lounge_chair	+Y
5	ahsap_masa	tables / desk	+Y
6	metal_uzun_dolap	storage / wardrobe	-Y
7	gm_55_ekran_tv_ret	storage / tv_stand	+Y
8	sf_af2f07d06f63	seating / office_chair	+Y
9	sf_a6cc78ad3aa2	kitchen / refrigerator	+Y
10	sf_62dbd2a0afb4	sketchfab / sofa	+Y

Otomatik kategorize akışı, dosya adı/malzeme örüntülerinden ve Blender başsız ölçümünden elde edilen verilerle bu varlıkların geometri ve semantik alanlarını doldurmuş; düşük güvenli alanlarda

Gemini ile zenginleştirme uygulanmıştır. Bu süreç, manuel etiketlemeye kıyasla hem hız hem de tutarlılık kazandırmıştır.

```
(.venv) PS C:\Users\ahaan\OneDrive\Desktop\Projects\asset_library\asset-library-mcp> python run_team.py --serve
Interior Design API starting...
API: http://localhost:8080
Docs: http://localhost:8080/docs
Open prompt_builder.html in your browser.

INFO: Started server process [15000]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8080 (Press CTRL+C to quit)
INFO: 127.0.0.1:57305 - "GET /health HTTP/1.1" 200 OK
INFO: 127.0.0.1:55155 - "GET /health HTTP/1.1" 200 OK
INFO: 127.0.0.1:62854 - "GET /health HTTP/1.1" 200 OK
INFO: 127.0.0.1:59632 - "GET /health HTTP/1.1" 200 OK
INFO: 127.0.0.1:51625 - "GET /health HTTP/1.1" 200 OK
INFO: 127.0.0.1:49727 - "GET /health HTTP/1.1" 200 OK
INFO: 127.0.0.1:51788 - "GET /health HTTP/1.1" 200 OK
INFO: 127.0.0.1:51600 - "GET /health HTTP/1.1" 200 OK
INFO: 127.0.0.1:55416 - "GET /health HTTP/1.1" 200 OK
INFO: 127.0.0.1:61326 - "GET /health HTTP/1.1" 200 OK
INFO: 127.0.0.1:61028 - "GET /health HTTP/1.1" 200 OK
INFO: 127.0.0.1:49284 - "GET /health HTTP/1.1" 200 OK
INFO: 127.0.0.1:51411 - "GET /health HTTP/1.1" 200 OK
INFO: 127.0.0.1:55846 - "GET /health HTTP/1.1" 200 OK
INFO: 127.0.0.1:52190 - "GET /materials/presets HTTP/1.1" 200 OK
INFO: 127.0.0.1:59044 - "GET /files/textures HTTP/1.1" 200 OK
INFO: 127.0.0.1:59044 - "GET /asset-manager/catalog HTTP/1.1" 200 OK
INFO: 127.0.0.1:59044 - "GET /thumbnails/sofa_leather_black_01.jpg HTTP/1.1" 304 Not Modified
INFO: 127.0.0.1:52190 - "GET /thumbnails/metal_office_desk_4k.jpg HTTP/1.1" 304 Not Modified
INFO: 127.0.0.1:64682 - "GET /thumbnails/mid_century_lounge_chair_4k.jpg HTTP/1.1" 304 Not Modified
INFO: 127.0.0.1:59044 - "GET /thumbnails/artichoke_red.jpg HTTP/1.1" 304 Not Modified
INFO: 127.0.0.1:52190 - "GET /thumbnails/ahsap_masa.jpg HTTP/1.1" 304 Not Modified
INFO: 127.0.0.1:52190 - "GET /thumbnails/metal_uzun_dolap.jpg HTTP/1.1" 304 Not Modified
INFO: 127.0.0.1:59044 - "GET /thumbnails/gm_55_ekran_tv_ret.jpg HTTP/1.1" 404 Not Found
INFO: 127.0.0.1:52190 - "GET /thumbnails/sf_af6c7ad83ae2.jpg HTTP/1.1" 304 Not Modified
INFO: 127.0.0.1:64682 - "GET /thumbnails/sf_af2f07d06f63.jpg HTTP/1.1" 304 Not Modified
INFO: 127.0.0.1:52190 - "GET /thumbnails/sf_c226b9a9cace.jpg HTTP/1.1" 200 OK
INFO: 127.0.0.1:59044 - "GET /thumbnails/sf_62dbd2a8afb4.jpg HTTP/1.1" 304 Not Modified
INFO: 127.0.0.1:65464 - "GET /blender/scene/analyze HTTP/1.1" 200 OK
INFO: 127.0.0.1:65464 - "GET /health HTTP/1.1" 200 OK
```

Şekil 6.3 Ajan adımlarına ait çalışma günlüğü (log) ve JSON çıktıları.

```
INFO: 127.0.0.1:52755 - "GET /sketchfab/search?q=floor%20lamp&category=interior-design HTTP/1.1" 200 OK
INFO: 127.0.0.1:57696 - "GET /prompt_builder.html HTTP/1.1" 200 OK
INFO: 127.0.0.1:57696 - "GET /ui/css/app.css HTTP/1.1" 304 Not Modified
INFO: 127.0.0.1:57696 - "GET /ui/js/state.js?v=20260530h HTTP/1.1" 304 Not Modified
INFO: 127.0.0.1:57696 - "GET /ui/js/materials.js?v=20260530h HTTP/1.1" 304 Not Modified
INFO: 127.0.0.1:57696 - "GET /ui/js/coach.js?v=20260530h HTTP/1.1" 304 Not Modified
INFO: 127.0.0.1:57696 - "GET /ui/js/scene.js?v=20260530h HTTP/1.1" 304 Not Modified
INFO: 127.0.0.1:57696 - "GET /ui/js/window-views.js?v=20260530h HTTP/1.1" 304 Not Modified
INFO: 127.0.0.1:57696 - "GET /ui/js/draw.js?v=20260530h HTTP/1.1" 304 Not Modified
INFO: 127.0.0.1:57696 - "GET /ui/js/models.js?v=20260530h HTTP/1.1" 304 Not Modified
INFO: 127.0.0.1:57696 - "GET /ui/js/sketchfab.js?v=20260530h HTTP/1.1" 304 Not Modified
INFO: 127.0.0.1:57696 - "GET /ui/js/sistem.js?v=20260531a HTTP/1.1" 304 Not Modified
INFO: 127.0.0.1:57696 - "GET /ui/js/app.js?v=20260530h HTTP/1.1" 304 Not Modified
INFO: 127.0.0.1:59801 - "POST /catalog/reload HTTP/1.1" 200 OK
INFO: 127.0.0.1:59801 - "GET /materials/presets HTTP/1.1" 200 OK
INFO: 127.0.0.1:59801 - "GET /blender/window-views/catalog HTTP/1.1" 200 OK
INFO: 127.0.0.1:59801 - "GET /catalog/houses HTTP/1.1" 200 OK
INFO: 127.0.0.1:59801 - "GET /asset-manager/catalog HTTP/1.1" 200 OK
INFO: 127.0.0.1:59801 - "GET /catalog/assets HTTP/1.1" 200 OK
INFO: 127.0.0.1:59801 - "GET /generated_views/Room_Wall_02_1780056528.png HTTP/1.1" 304 Not Modified
INFO: 127.0.0.1:59801 - "GET /generated_views/Room_Wall_05_1780010601.png HTTP/1.1" 304 Not Modified
INFO: 127.0.0.1:59801 - "GET /favicon.ico HTTP/1.1" 404 Not Found
INFO: 127.0.0.1:57696 - "GET /health HTTP/1.1" 200 OK
INFO: 127.0.0.1:57696 - "GET /sketchfab/search?q=floor%20lamp&category= HTTP/1.1" 200 OK
INFO: 127.0.0.1:54027 - "POST /sketchfab/download HTTP/1.1" 200 OK
INFO: 127.0.0.1:56627 - "GET /health HTTP/1.1" 200 OK
INFO: 127.0.0.1:56627 - "GET /sketchfab/job/sf_ca057759 HTTP/1.1" 200 OK
INFO: 127.0.0.1:56627 - "GET /sketchfab/job/sf_ca057759 HTTP/1.1" 200 OK
INFO: 127.0.0.1:56627 - "GET /sketchfab/job/sf_ca057759 HTTP/1.1" 200 OK
INFO: 127.0.0.1:56627 - "GET /sketchfab/job/sf_ca057759 HTTP/1.1" 200 OK
INFO: 127.0.0.1:56627 - "GET /sketchfab/job/sf_ca057759 HTTP/1.1" 200 OK
INFO: 127.0.0.1:56627 - "GET /asset-manager/catalog HTTP/1.1" 200 OK
INFO: 127.0.0.1:56627 - "GET /thumbnails/gm_55_ekran_tv_ret.jpg HTTP/1.1" 404 Not Found
INFO: 127.0.0.1:56627 - "GET /thumbnails/sf_7044aedec91b.jpg HTTP/1.1" 200 OK
INFO: 127.0.0.1:56627 - "GET /health HTTP/1.1" 200 OK
```

Şekil 6.4 add_asset.py otomatik kategorize terminal çıktısı.

6.5. Niceliksel Değerlendirme

Sistemin en kritik bileşeni olan yerleştirme/geometri motoru (`calculate_furniture_layout`), büyük dil modeli, API anahtarı ve Blender gerektirmeden — yalnızca saf Python algoritma katmanını olarak — niceliksel biçimde ölçülmüştür. Ölçümler, **katalogdaki gerçek varlıklar** (masa, ofis sandalyesi, kanepe, TV ünitesi, gardırop) kullanılarak beş yerleştirme senaryosunda Python 3.13 ortamında yapılmıştır. Her senaryoda işlem süresi 30 tekrarın ortalaması olarak ölçülmüş;

çıkıtının oda iç sınırı içinde kalıp kalmadığı (taşma kontrolü) ve aynı girdinin aynı çıktıyı üretip üretmediği (20 tekrarla determinizm) doğrulanmıştır.

Tablo 6.2 Algoritma katmanı niceliksel ölçüm sonuçları (ortam: saf Python 3.13; LLM/Blender hariç).

Senaryo	Oda (W×D, m)	Yerleştirme	Süre (ort, ms)	Oda içinde	Determinizm
A1 — Ofis	5 × 4	masa (kuzey duvar) + sandalye (masa önü)	0,359	2/2	✓
A2 — Oturma odası	4 × 3	kanepe (güney duvar) + TV (kuzey duvar)	0,361	2/2	✓
A3 — Dar oda (patolojik)	2 × 2	masa (kuzey duvar) + sandalye	0,360	0/2*	✓
A4 — Köşe yerleşim	5 × 5	kanepe (KB köşe) + gardırop (doğu duvar)	0,372	2/2	✓
A5 — Merkez	6 × 4	masa (merkez)	0,188	1/1	✓

*A3, masa genişliğinin (2,0 m) oda iç genişliğinden ($\approx 1,8$ m) büyük olduğu, **bilerek seçilmiş patolojik** bir durumdur: masa fiziksel olarak odadan geniş olduğu için iki yandan taşar ve ona bağlı sandalye geçerli bir konuma yerleştirilemez. Bkz. Bölüm 6.6.

Elde edilen başlıca sonuçlar:

- **İşlem süresi:** Yerleşim hesabı, istek başına ortalama $\approx 0,33$ ms sürmektedir (tekli mobilyalı en yalın senaryo A5'te 0,188 ms; iki mobilyalı senaryolarda $\approx 0,36$ ms). Yani algoritma katmanı sistemin darboğazı değildir; toplam yanıt süresine asıl katkı, büyük dil modeli çağrılarından gelir.
- **Determinizm (NFR-4):** Beş senaryonun **tamamında**, 20'şer tekrarda aynı girdi bayt-bayt aynı çıktıyı üretmiştir (her senaryoda farklı çıktı sayısı = 1/20). Bu, algoritma katmanının tümüyle deterministik olduğunu doğrular.
- **Taşma kontrolü (NFR-5):** Mobilyaların oda iç boyutlarına sığıdığı dört normal senaryoda (A1, A2, A4, A5) yerleştirilen **7 mobilyanın 7'si de (%100)** oda iç hacmi içinde kalmıştır. Yalnızca, mobilyanın odadan fiziksel olarak geniş olduğu patolojik A3 senaryosunda taşma gözlenmiştir (aşağıda tartışılmıştır).

6.6. Bilinen Sınırlar

Geliştirilen sistem, doğal dil ile 3B sahne kurgusu arasındaki boşluğu başarıyla kapatmakla birlikte, mevcut sürümün bazı sınırları bulunmaktadır:

Tablo 6.3 Bilinen sınırlar ve etkileri.

Sınır	Etki / Açıklama
Mobilya odadan fiziksel olarak büyükse taşma	İç hacim kırpma mobilyanın <i>konumunu</i> oda sınırına çeker; ancak mobilya genişliği oda iç genişliğinden büyükse (ör. 2,0 m masa, $\approx 1,8$ m iç genişlikli oda) nesne iki yandan taşar ve ona bağlı nesne yerleştirilemez. Bölüm 6.5'teki A3 senaryosu bu durumu gösterir. Taşmasızlık garantisi "mobilya oda iç boyutlarına sığıldığı sürece" geçerlidir.
Henüz container (bina) varlığı yok	Workflow B kod düzeyinde hazırdir ancak gerçek bir bina varlığıyla uçtan uca doğrulanmamıştır
PCA ön yön tespiti simetrik nesnelerde belirsiz	Kare/yuvarlak masalarda güven düşer; kullanıcı onayı (güvenlik ağı) devreye girer
<code>interior_bbox_local</code> tek kutu	L biçimli odalar için yetersiz; çok-poligonlu iç hacim gerekir
Sandalye yön düzeltmesi uç durumu	<code>forward_axis</code> ile bağlı dönüşün birlikte çalışması <code>facing_correction_z</code> doğru ayarlanmazsa çift dönüşü yol açabilir
Blender yolu (PATH) bağımlılığı	Başsız çağrılar Blender çalıştırılabilirini arar; farklı işletim sistemlerinde yol tespiti tamamlanmalı
Birim test kapsamı yok	Doğrulama uçtan uca, elle (web arayüzü + Blender) yapılır
Dil modeli tutarlılığı	<code>temperature=0</code> kullanılsa da farklı çalıştırmalarda varyans olabilir; few-shot örneklerle iyileştirilebilir

Bu sınırların önemli bir kısmı, mimarinin temelini değiştirmeden, veri katmanı (daha fazla varlık ve bina modeli) ve hesaplama katmanı (çok-poligonlu iç hacim, daha sağlam yön tespiti) iyileştirmeleriyle aşılabilecek niteliktedir.

7. SONUÇLAR

Bu proje çalışmasında, kullanıcının doğal dil ile ifade ettiği iç mekân tasarım isteklerini otomatik olarak üç boyutlu bir Blender sahnesine dönüştüren, çok-ajanlı yapay zekâ tabanlı bütünleşik bir sistem tasarlanmış, geliştirilmiş ve uygulanmıştır. Sistem; mekân analizi, mobilya seçimi, uzamsal yerleştirme ve Blender'da yürütme görevlerini ayrı uzman ajanlara dağıtan dört katmanlı bir işlem hattı üzerine kurulmuştur.

Çalışmanın başlıca katkıları şunlardır:

- **Anlamsal olarak zenginleştirilmiş v2 katalog şeması**, varlıkları yalnızca sınıflandırmakla kalmayıp; oda uyumu, eş varlık ilişkileri, taban ayak izi ve yönelim bilgileriyle birlikte tanımlayarak ajanların akıllı seçim yapmasını mümkün kılmıştır.
- **İç hacim sınırlayıcı kutu kırpma ve kırpmanın bağıl yerleşimlere yansıtılması**, mobilyaların oda/bina dışına taşmasını engellerken nesneler arası görelî geometriyi (örneğin masaya dönük sandalye) korumuştur.
- **Hibrit otomatik kategorize** akışı; dosya adı/malzeme örüntülerini, Blender ile başsız ölçümü, temel bileşen analizini ve dil modeli zenginleştirmesini birleştirerek yeni varlıkların kataloğa tutarlı biçimde eklenmesini sağlamıştır.
- **Sağlayıcıdan bağımsız** tasarım, sistemin hem Gemini hem Claude modelleriyle çalışabilmesine olanak tanımıştır.

Sonuç olarak ortaya konan sistem; iç mimari konseptlerinin doğal dil ile hızlı prototiplenmesine olanak veren, modüler, genişletilebilir ve teknik bilgi engelini ortadan kaldıran bir altyapıdır.

7.1. Gelecek Çalışmalar

Sistemin gelecekteki gelişimi için öngörülen başlıca yönler:

- **Bina (container) varlık kütüphanesinin oluşturulması** ve Workflow B'nin gerçek bina modelleriyle uçtan uca doğrulanması.
- **Çok-poligonlu iç hacim** desteği ile L biçimli ve düzensiz oda geometrilerinin doğru kırılması.
- **Daha sağlam ön yön tespiti** (PCA'nın yanı sıra geometri/öğrenme tabanlı yöntemlerle simetrik nesnelerin yöneliminin çözülmesi).
- **Fotogerçekçi görselleştirme** (otomatik kamera, ışık ve malzeme/HDRI ortam kurulumu ile render alma).
- **Sıfırdan modelleme** vizyonu: hazır modellerle dekorasyondan, yapay zekâ ajanlarının Blender ile özgün geometri üretmesine geçiş.
- **Birim ve entegrasyon testleri** ile sistem güvenilirliğinin artırılması.

KAYNAKLAR

- [1] T. B. Brown et al., "Language Models are Few-Shot Learners," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 1877–1901, 2020.
- [2] H. Lee, M. Savva, ve A. X. Chang, "Text-to-3D Shape Generation," *Computer Graphics Forum (State-of-the-Art Report)*, 2024. arXiv:2403.13289. Erişilebilir: <https://arxiv.org/abs/2403.13289>.
- [3] K. Fukamizu, M. Kondo, ve T. Sakamoto, "Generation High Resolution 3D Model from Natural Language by Generative Adversarial Network," arXiv:1901.07165, 2019. Erişilebilir: <https://arxiv.org/abs/1901.07165>.
- [4] A. Sanghi, R. Fu, V. Liu, K. D. D. Willis, H. Shayani, A. H. Khasahmadi, S. Sridhar, ve D. Ritchie, "CLIP-Sculptor: Zero-Shot Generation of High-Fidelity and Diverse Shapes from Natural Language," *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2023. arXiv:2211.01427. Erişilebilir: <https://arxiv.org/abs/2211.01427>.
- [5] C. Li, C. Zhang, J. Zhou, ve ark., "Generative AI meets 3D: A Survey on Text-to-3D in AIGC Era," arXiv:2305.06131, 2023. Erişilebilir: <https://arxiv.org/abs/2305.06131>.
- [6] B. Wen, H. Xie, Z. Chen, F. Hong, ve Z. Liu, "3D Scene Generation: A Survey," arXiv:2505.05474, 2025. Erişilebilir: <https://arxiv.org/abs/2505.05474>.
- [7] M. Wooldridge, *An Introduction to MultiAgent Systems*, 2nd ed. Chichester, U.K.: John Wiley & Sons, 2009.
- [8] S. Ramírez, "FastAPI Documentation," fastapi.tiangolo.com.
- [9] Agno (agno-agi), "Agno: Multi-Agent Framework Documentation," github.com/agno-agi/agno.
- [10] Google, "Gemini API / google-genai SDK Documentation," ai.google.dev.
- [11] Anthropic, "Claude API and Python SDK Documentation," docs.anthropic.com.
- [12] Blender Foundation, "Blender Python API (bpy) Documentation," docs.blender.org.
- [13] I. T. Jolliffe, *Principal Component Analysis*, 2nd ed. New York, NY, USA: Springer, 2002.
- [14] Anthropic, "Model Context Protocol (MCP) Specification," modelcontextprotocol.io.
- [15] Encode, "Uvicorn: ASGI Web Server Documentation," uvicorn.dev.
- [16] Encode, "HTTPX Documentation," python-httpx.org.
- [17] Pydantic, "Pydantic Documentation," docs.pydantic.dev.